

DD: WO-01-FLOW-SHIFTING — Shifting (Relocation) Work Order Flow

Developer implementation guide: DD_WO-01-FLOW-SHIFTING-DEV_IMPL_GUIDE-v1.0.md.

1. Purpose

This rewritten DD is the developer-facing version of the operator BPMN flow DD_WO-01-FLOW-SHIFTING-v1.0.md.

Verdict: ■ New | **Wave:** 2 | **Group III:** Fulfillment & Operations | **Version:** v1.0 | **Module:** Work Order | **Satellite of:** DD_WO-01-Work_Order_Module-v1.0 The Shifting flow satellite. Defines `workOrder_Shifting_WIK.bpmn`, the Drools KJAR shape for the disconnect-first / reconnect-second sequence, the operator-config rows seeding the framework's catalogs, the single-WO two-phase mechanism (`current_phase: DISCONNECT → RECONNECT` within one process instance), the new-address capture, the HomePass topology resolution at reconnect-finalize, and the account-status side-effects (INA at disconnect, STF at reconnect). Built on DD_WO-01-FRAMEWORK-v1.0; relies on its primitives without re-stating them. Covers all Job Types in the SHIFTING kind: GSD (GPON Shifting Disconnect), GSR (GPON Shifting Reconnect), HSD (HFC Shifting Disconnect), HSR (HFC Shifting Reconnect). The `job_type_code` on the WO row transitions from *SD to *SR when the BPMN phase transitions — one `wo_id` throughout, one process instance throughout.

The goal of this rewrite is to make the DD easier for a mid-level developer to implement without losing the detailed source contract.

2. Implementation Scope

This DD should be implemented as part of the owning SOPHIX capability, not as an isolated service unless the deployment architecture explicitly separates that capability.

Implement this DD with these rules:

- Use the owning module APIs/repositories for authoritative writes.
- Use approved internal APIs/client wrappers when reading another module's data.
- Do not query another module's database tables directly from business flow code.
- Treat worker names as logical Camunda external-task topics, not separate deployments.
- One worker application may handle multiple topics, but metrics, retries, and logs must remain visible per topic.
- Emit success events only after the authoritative state commit succeeds.
- Use outbox publishing for Kafka events.

3. Runtime Metadata

Item	Value
Source file	/Users/macbook/Downloads/Sophix_V3_BSS_DDs_MD_2026-05-27/07_work_order/DD_WO-01-FLOW-SHIFTING-v1.0.md
Version	v1.0
DD type	operator BPMN flow
BPMN/process keys	Not explicitly defined
Drools packages	Not explicitly defined

4. Runtime Contracts To Implement

4.1 APIs

- POST /api/work-orders
- POST /api/work-orders/wo_01HZ...SH7/finalize-first-confirm
- POST /api/work-orders/wo_01HZ...SH7/finalize-second-confirm
- POST /api/work-orders/wo_01HZ...SH7/notes

- POST /api/work-orders/{woId}/cancel
- POST /api/work-orders/{woId}/phase-events-replay
- POST /claim

4.2 Tables

- No CREATE TABLE block is explicitly declared in this DD.

For each table in this DD, implement the schema with:

- a short table comment explaining what the table is for;
- column comments or migration notes explaining each field;
- constraints and indexes from the DD;
- seed data where the DD provides operator-specific sample rows.

4.3 Worker Topics

- await-csr-confirm-disconnect
- client-up-decision
- contractor-assignment-decision
- r1-vip-internal-first
- r2-hfc-by-region
- r3-gpon-by-region
- r4-fallback-default
- wo-auto-assign-contractor
- wo-capture-equipment-bindings-at-finalize
- wo-emit-homepass-update-at-finalize
- wo-emit-shifting-completed
- wo-enforce-finalize-checklist
- wo-mark-pattern-b-reroute
- wo-shifting-disconnect-prepare
- wo-shifting-enter-disconnect
- wo-shifting-enter-reconnect
- wo-shifting-finalize-disconnect-phase
- wo-shifting-finalize-reconnect-phase
- wo-validate-create

Worker-topic guidance:

- A BPMN service task uses the topic.
- A worker application subscribes to the topic.
- The topic handler validates input variables, calls the owning module/API, writes outputs back to Camunda, and records metrics.
- Do not treat the topic string as a shell command or Kubernetes deployment name.

4.4 Events

- WorkOrderAssigned
- WorkOrderCancellationRequested
- WorkOrderClaimedByTechnician

- WorkOrderCreated
- WorkOrderEquipmentBindingsRecorded
- WorkOrderFinalized
- WorkOrderHomePassDetachment
- WorkOrderHomePassTopologyResolved
- WorkOrderHomePassVacating
- WorkOrderInProgress
- WorkOrderNoteAppended
- WorkOrderPhaseFinalized
- WorkOrderPhaseTransitioned
- WorkOrderReroutedPatternB
- WorkOrderShiftingCompleted

Event guidance:

- Write events through the transactional outbox.
- Include operation id, aggregate id, operator code, actor, and correlation data.
- Consumers should be able to rebuild downstream state without querying workflow internals.

5. Developer Implementation Checklist

1. Add or verify database migrations and seed rows.
2. Add or verify config rows for the operator/module.
3. Implement request/command DTOs and validation.
4. Implement idempotency and in-flight operation checks where the DD requires them.
5. Implement Drools fact mapping and package deployment where rules are used.
6. Implement BPMN process, service tasks, gateways, timers, and message catches where the DD is a flow.
7. Implement worker-topic handlers using owning module APIs.
8. Implement compensation paths and manual recovery paths.
9. Emit success/rejected events through the outbox.
10. Add smoke tests for happy path, validation failure, dependency failure, and retry/idempotency.

6. Infrastructure And AWS Notes

Deploy this DD with the existing SOPHIX platform components unless the owning module requires isolation:

Concern	Recommendation
API/runtime	Run inside the owning module deployment or existing workflow API pods.
Workers	Consolidate low-volume topics into shared worker apps; keep per-topic metrics.
Database	Use the owning module PostgreSQL schema and migration pipeline.
Kafka	Use the foundation Kafka/MSK setup and transactional outbox.
Camunda	Deploy BPMN files through the workflow deployment pipeline.
Drools	Deploy KJAR/rule artifacts through the approved rules pipeline.
Secrets	Use the platform secret manager; on AWS prefer Secrets Manager/Parameter Store with IRSA.
Observability	Alert on stuck operations, failed workers, outbox backlog, and external dependency failures.

7. Original DD Detail Preserved

The following section preserves the detailed source contract for implementation and review.

Verdict: ■ New | **Wave:** 2 | **Group III:** Fulfillment & Operations | **Version:** v1.0 | **Module:** Work Order | **Satellite of:** DD_WO-01-Work_Order_Module-v1.0

The Shifting flow satellite. Defines `WorkOrder_Shifting_WIK.bpmn`, the Drools KJAR shape for the disconnect-first / reconnect-second sequence, the operator-config rows seeding the framework's catalogs, the single-WO two-phase mechanism (`current_phase: DISCONNECT → RECONNECT` within one process instance), the new-address capture, the HomePass topology resolution at reconnect-finalize, and the account-status side-effects (INA at disconnect, STF at reconnect). Built on DD_WO-01-FRAMEWORK-v1.0; relies on its primitives without re-stating them.

Covers all Job Types in the SHIFTING kind: GSD (GPON Shifting Disconnect), GSR (GPON Shifting Reconnect), HSD (HFC Shifting Disconnect), HSR (HFC Shifting Reconnect). The `job_type_code` on the WO row transitions from *SD to *SR when the BPMN phase transitions — one `wo_id` throughout, one process instance throughout.

1. What this flow is

A Shifting WO covers one business operation: a customer is staying with Wananchi but relocating. The old address must be vacated (drop returned to inventory or recovered in place, account inactivated) and the new address must be served (new drop spliced or activated, account reactivated at the new location). The two halves are tightly coupled by business rule: the disconnect must finalize before the reconnect can start, and the reconnect's finalization is what updates the HomePass and reactivates the account at the new place. Shifting is free for the customer.

The framework's multi-phase BPMN mechanism is built precisely for this case. The flow uses a single `wo_id` with `current_phase` cycling `NULL → DISCONNECT → RECONNECT` over the life of one process instance. Same WO, same audit trail, finalization-order rule enforced naturally by BPMN sequence.

Three non-negotiable rules from the workshop:

1. **Finalization order is sequential, not parallel.** Disconnect finalizes first; reconnect cannot start until then. The BPMN enforces this with a single-thread sequence (no parallel gateway).
2. **The disconnect phase has no site visit.** GSD/HSD are system-driven — the network simply turns off provisioning at the old address. No technician dispatched. The framework's `wo_job_type_catalog.requires_site_visit = false` for these codes; the BPMN takes the no-site-visit fast path.
3. **The reconnect phase is a full installation-like flow.** Tech dispatched, on-site work, optical check, equipment binding (where the relocation involves new equipment), HomePass topology resolution at finalize. Same primitives as the Installation flow uses.

This BSS is deployable in any country Wananchi Group operates. The cross-country variation that affects Installation (§1 of DD_WO-01-FLOW-INSTALLATION) applies equally here:

- **In Kenya (WIK)**, the new address's HomePass record carries OLT port + PBO + splitter details from the network-planning team. The reconnect tech *confirms* this topology when finalizing — or *corrects* it if reality differs from planning.
- **In Senegal (YASSN)**, the new address's HomePass record at relocation time may have only the address + upstream anchor; if the last mile to the new premise isn't already deployed, the reconnect tech *captures* the topology via "app terrain" mobile app — same mechanism as a fresh install.

What this flow does NOT do

- **Doesn't handle non-Wananchi-staying disconnects.** Termination, suspension, churn — those are separate flows (future OSR-DISCONNECT). Shifting is specifically *the customer stays with us, just moves*.
- **Doesn't handle network migration.** HFC → GPON migration (job codes GPO, HHD) is a different kind of operation — the customer doesn't physically move, only the technology serving them changes. Future flow.
- **Doesn't own contractor invoicing for shifting visits.** Same as Installation — Finance consumes `WorkOrderShiftingCompleted` (§8) for invoicing per the contractor's shifting-fee arrangement (separate from install fee schedules).
- **Doesn't bill the customer.** Shifting is free of charge. No coupling to BIL-.*
- **Doesn't define the WO module's state machine, REST API, schemas, contractor model, or Kafka primitives** — those live in DD_WO-01-FRAMEWORK. This satellite extends the framework with shifting-specific Camunda BPMN, Drools rules, operator config rows, and the phase-transition semantics.

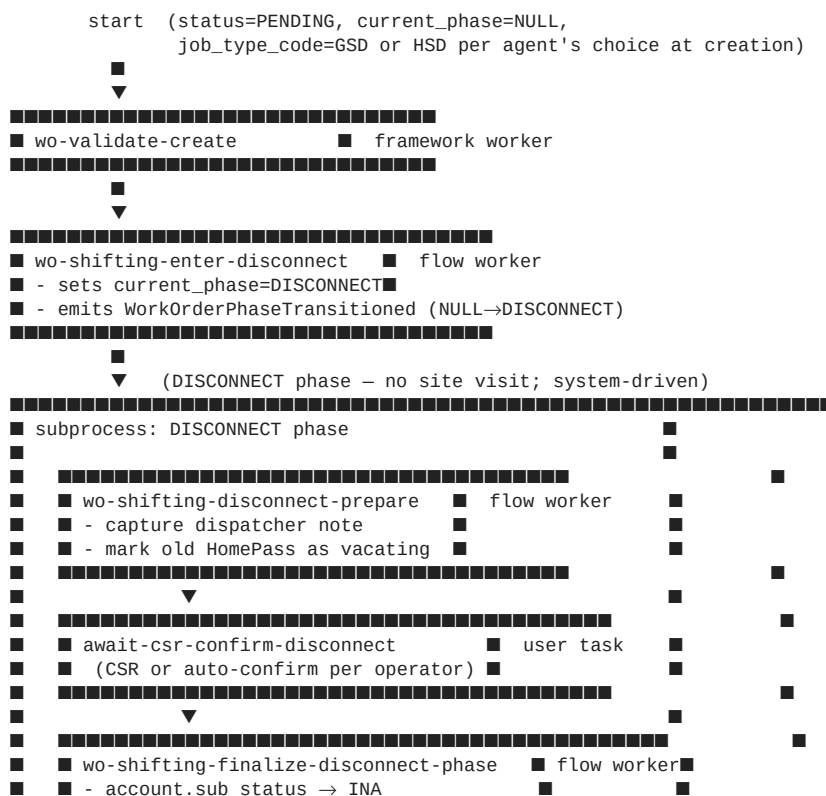
Glossary additions (specific to this flow)

Term	Meaning
Phase	A sub-state within status=IN_PROGRESS. The Shifting flow uses two phases: DISCONNECT (the GSD/HSD half) and RECONNECT (the GSR/HSR half). Transitions are recorded on wo_status_history (with previous/new phase columns) and emitted as WorkOrderPhaseTransitioned Kafka events.
Single-WO two-phase	The shifting model where one wo_id carries the entire customer relocation. Contrast with a hypothetical "paired-WO" model where GSD and GSR would be two separate WO rows linked by master_wo_id. Single-WO is the chosen v1.0 design — simpler audit trail, finalization-order enforced by BPMN, no cross-WO coordination needed.
Old HomePass	The HomePass record at the customer's pre-shifting address. Read by the disconnect phase; updated at disconnect-finalize to mark the customer no longer associated.
New HomePass	The HomePass record at the customer's post-shifting address. Captured at WO creation (the agent selecting the new address); updated at reconnect-finalize with confirmed or captured topology.
Account status side-effects	The two account-status changes that happen as part of a Shifting WO: at disconnect-finalize, sub_status → INA (Inactive); at reconnect-finalize, sub_status → STF (Staff/active). Both are emitted as event-payload fields on WorkOrderFinalized (or on WorkOrderPhaseFinalized — see §8) for ILM-CFG-01 to consume.
New-address capture	The customer's new physical address selected at WO creation by the CSR (or by the contractor team if they raise the shifting). Stored as new_homepass_id in WO metadata. The reconnect phase dispatches to this address.
App terrain (cross-country)	Same as in DD_WO-01-FLOW-INSTALLATION glossary. The Senegal field-tech mobile app; from the WO module's perspective, just another REST client. Used to capture topology during the GSR/HSR reconnect phase where the last mile is deployed during the shifting itself.

2. BPMN — WorkOrder_Shifting_WIK.bpmn

One BPMN per operator (v1.0 ships WIK only). The process key is WorkOrder_Shifting_WIK; future operators get WorkOrder_Shifting_WUG, _WTZ, _YASSN. Same workers run; per-operator variation is in operator-config rows (§3) + the operator's Drools KJAR (§4).

2.1 Process shape




```

end

+ boundary message catch (interrupting): WorkOrderCancellationRequested
  → wo-mark-cancelled (framework worker) → status=CANCELLED → end-cancelled

```

The flow has two subprocesses (DISCONNECT and RECONNECT) chained sequentially within one process instance. The reconnect phase's middle section mirrors the Installation flow (assignment → claim → start → work loop → 2-step finalize → Drools client-up gate → Pattern B option). The differences from Installation are the explicit phase-entry workers, the disconnect-phase wrapper around all of it, and the final HomePass update going to the **new** address rather than confirming the existing one.

2.2 BPMN XML — phase-transition snippet

The phase transitions use a flow service task (not the REST /transition-phase API) because the transition is internal — the BPMN itself drives it, no external caller needs to push.

```

<bpmn:serviceTask
  id="wo-shifting-enter-reconnect"
  name="Enter RECONNECT phase"
  camunda:type="external"
  camunda:topic="wo-shifting-enter-reconnect">
  <bpmn:extensionElements>
    <camunda:inputOutput>
      <camunda:inputParameter name="woId">${woId}</camunda:inputParameter>
      <camunda:inputParameter name="targetPhase">RECONNECT</camunda:inputParameter>
      <camunda:inputParameter name="updateJobTypeCode">true</camunda:inputParameter>
    </camunda:inputOutput>
  </bpmn:extensionElements>
</bpmn:serviceTask>

```

The worker (§7.2) calls the framework's internal phase-update service (same code path the public /transition-phase REST API would use), and additionally updates `job_type_code` from GSD → GSR (or HSD → HSR) based on the WO's network type.

2.3 BPMN XML — cancellation boundary

Same framework pattern as DD_WO-01-FLOW-INSTALLATION §2.2 — interrupting boundary message event on `WorkOrderCancellationRequested` attached to the top-level subprocess. Cancellation is valid at any phase. If cancelled during DISCONNECT, the account stays as it was before the WO; if cancelled during RECONNECT, account stays in INA (the disconnect already finalized), and ops must follow up to either re-issue a reconnect WO or terminate the customer.

2.4 Per-operator BPMN composition

The Shifting flow can be composed differently per operator without code change. v1.0 ships WIK. Future operators can:

- Make the DISCONNECT phase require dispatcher confirmation (WIK auto-confirms after the prepare worker; YASSN might require a CSR step).
- Insert a pre-disconnect equipment-recovery check (some operators require the modem returned before disconnect-finalize; others recover the equipment at reconnect at the new address; others ignore equipment recovery during shifting).
- Tune the RECONNECT phase's Drools client-up thresholds and Pattern B routing rules.
- Add intermediate user tasks for compliance approvals or VIP-customer handling.

All of these are BPMN edits and config-row additions, not framework or code changes.

3. Per-operator + per-flow configuration rows

This flow seeds the following framework catalogs at v1.0 deploy time. All rows are operator-scoped to WIK.

3.1 wo_flow_config row

operator_code	kind	bpmn_process_key	drools_kjar_ref
WIK	SHIFTING	WorkOrder_Shifting_WIK	wo-shifting-wik-v1.0.kjar

3.2 wo_job_type_catalog rows

operator_code	job_type_code	kind	display_name	description	network_type	requires_site_visit	warranty_days
WIK	GSD	SHIFTING	GPON Shifting Disconnect	Disconnect GPON client at old address during a relocation	GPON	false	0
WIK	GSR	SHIFTING	GPON Shifting Reconnect	Reconnect GPON client at new address — new address selected during this WO	GPON	true	0
WIK	HSD	SHIFTING	HFC Shifting Disconnect	Disconnect HFC client at old address during a relocation	HFC	false	0
WIK	HSR	SHIFTING	HFC Shifting Reconnect	Reconnect HFC client at new address after HSD	HFC	true	0

Note: warranty_days = 0 for all shifting codes. The customer's original install warranty (from the initiating install WO 6 months earlier) doesn't extend by relocating; shifting is a coordination operation, not a fresh installation.

3.3 wo_note_kind_registry rows (shifting-specific kinds)

Inherits framework's generic kinds + reuses installation-flow-registered kinds (findings, solution, optical_readings, hfc_signal_readings, client_up_evidence, network_topology_capture, network_topology_confirmation). Adds:

operator_code	note_kind	schema_jsonb (excerpt)	append_only
WIK	new_address	{"required":["newHomepassId","addressFreeText"],"properties":{"newHomepassId":{"type":"string"},"addressFreeText":{"type":"string"},"agentNotes":{"type":"string"}}}	false
WIK	shifting_disconnect_summary	{"required":["actionTaken"],"properties":{"actionTaken":{"enum":["PROVISIONING_DISABLED_REMOTELY","EQUIPMENT_RECOVERY_PENDING","CUSTOMER_KEEP_EQUIPMENT"]}}}	true

The new_address note kind is non-append-only: if the customer changes their mind about the new address before the reconnect phase begins, the agent can supersede the prior new_address note with an updated one (framework's superseded_by_note_id pattern).

3.4 wo_finalization_requirements rows

Two rows per kind — one per phase, keyed by job_type_code. The framework's wo-enforce-finalize-checklist worker reads the row matching the CURRENT job_type_code (which is GSD/HSD during disconnect-phase finalize, GSR/HSR during reconnect-phase finalize).

operator_code	kind	job_type_code	required_note_kinds	required_attachment_categories	min_attachments_per_category
WIK	SHIFTING	GSD	["dispatcher_note","shifting_disconnect_summary","final_reason_set"]	[]	NULL
WIK	SHIFTING	HSD	["dispatcher_note","shifting_disconnect_summary","final_reason_set"]	[]	NULL

operator_code	kind	job_type_code	required_note_kinds	required_attachment_categories	min_attachments_per_category
WIK	SHIFTING	GSR	["findings", "solution", "optical_readings", "client_up_evidence", "new_address", "network_topology_confirmation", "final_reason_set"]	["setup_photo", "rf_optical_reading"]	NULL
WIK	SHIFTING	HSR	["findings", "solution", "hfc_signal_readings", "client_up_evidence", "new_address", "network_topology_confirmation", "final_reason_set"]	["setup_photo", "speed_test", "rf_optical_reading"]	NULL

For YASSN (when its flow ships), the GSR/HSR rows will use `network_topology_capture` instead of `network_topology_confirmation`, mirroring the cross-country variation documented in DD_WO-01-FLOW-INSTALLATION §3.

3.5 wo_final_reason_catalog rows

operator_code	kind	job_type_code	final_reason_code	display_name	triggers_escalation_event
WIK	SHIFTING	GSD	DISCONNECTED_REMOTELY	Disconnected remotely (provisioning disabled)	false
WIK	SHIFTING	GSD	CUSTOMER_CANCELLED_RELOCATION	Customer cancelled relocation before disconnect	false
WIK	SHIFTING	HSD	DISCONNECTED_REMOTELY	Disconnected remotely (provisioning disabled)	false
WIK	SHIFTING	HSD	CUSTOMER_CANCELLED_RELOCATION	Customer cancelled relocation before disconnect	false
WIK	SHIFTING	GSR	RECONNECTED_AT_NEW_ADDRESS	Reconnected at new address, customer up	false
WIK	SHIFTING	HSR	RECONNECTED_AT_NEW_ADDRESS	Reconnected at new address, customer up	false
WIK	SHIFTING	GSR	NEW_ADDRESS_NOT_SERVICEABLE	New address not serviceable – refer to retention	false
WIK	SHIFTING	HSR	NEW_ADDRESS_NOT_SERVICEABLE	New address not serviceable – refer to retention	false

Note: `NEW_ADDRESS_NOT_SERVICEABLE` is the "shifting failed at reconnect" outcome. It does not escalate via QCS (that's a Support flow concern); it ends the shifting WO in `COMPLETED` status with a final-reason that retention picks up via Kafka subscription to spawn a retention conversation with the customer.

3.6 Operator variation surface

The same six axes documented in DD_WO-01-FLOW-INSTALLATION §3.6 apply here: BPMN composition, Drools KJAR, `wo_job_type_catalog`, `wo_note_kind_registry`, `wo_finalization_requirements`, `wo_final_reason_catalog`. The Shifting flow does not introduce new variation axes.

Cross-country instance for SN: YASSN registers `network_topology_capture` (instead of `network_topology_confirmation`) for GSR/HSR finalization requirements, and the GSR/HSR `wo_finalization_requirements` row references it. The mobile app posts the captured topology via the same framework REST API; the `WorkOrderHomePassTopologyResolved` event carries `topologyMode=CAPTURED` and `RLM-CFG-01`

writes the captured fields onto the NEW homepass record at reconnect-finalize.

4. Drools KJAR — wo-shifting-wik-v1.0.kjar

The flow's KJAR holds two named decisions:

1. **contractor-assignment-decision** — same shape as the Installation flow's (§4.1 there). Picks contractor + team + (optional) tech for the RECONNECT phase. The DISCONNECT phase does NOT consult this decision because no site visit is needed.
2. **client-up-decision** — same shape as the Installation flow's (§4.2 there). Decides whether RECONNECT-phase finalization proceeds to second-confirm or routes to Pattern B reroute.

4.1 contractor-assignment-decision — RECONNECT phase only

Inputs: {operatorCode, kind=SHIFTING, jobTypeCode=GSR|HSR, networkType, techRegionCode (from NEW HomePass), customerVipTier?, scheduledAt}.

Output: ContractorAssignmentDecision { contractorId, teamId, technicianId?, mode, considered[] }.

v1.0 KE WIK rules — same shape as Installation §4.1 but applied to the NEW HomePass's tech_region_code:

Rule	Trigger	Decision
r1-vip-internal-first	customerVipTier == "VIP"	Prefer ctr_Z071H regardless of network/region.
r2-hfc-by-region	networkType == "HFC" and new-address tech region in WIK in-house coverage list	ctr_Z071H TEAM mode.
r3-gpon-by-region	networkType == "GPON" and new-address tech region in WIK in-house coverage list	ctr_Z071H TEAM mode.
r4-fallback-default	Otherwise	ctr_ADRIAN TEAM mode (external).

The shifting reconnect doesn't need different routing logic from a fresh install — the work shape (run drop, splice, install ONT/modem, verify customer up) is the same. The operator's KJAR may contain the rules directly in this file, or import from the installation KJAR for reuse — implementation detail.

4.2 client-up-decision — RECONNECT phase

Same as Installation §4.2. Inputs: {woId, jobTypeCode (GSR or HSR), finalReasonCode, latestSignalReading or latestOpticalReading, attachmentCategories[]}. Output: CLIENT_UP or CLIENT_NOT_UP_REROUTE.

Thresholds: same as Installation flow (HFC downstream $\in [-15,15]$ dBm, SNR ≥ 30 dB; GPON ontRxDbm $\in [-28,-8]$ dBm). Reasoning: if the customer is at a new address with new last-mile work but the readings don't pass, the right operational response is the same — reroute to maintenance or construction to fix the line.

5. Phase transition mechanics

5.1 Single-WO two-phase model

Workshop documents two distinct WO codes (GSD/GSR for GPON, HSD/HSR for HFC) and a strict finalization order. We model these as one WO row with current_phase cycling, not two paired WOs. The decision (settled during framework design):

- **One audit trail.** Searching the WO history shows the complete shifting story in one place.
- **Finalization order enforced by BPMN sequence,** not by cross-WO coordination logic. If the disconnect subprocess hasn't completed, the reconnect subprocess can't run.
- **No master/sub linkage needed.** The master_wo_id primitive (framework §5.1) is reserved for Ticketing-driven escalations and RPT linkages.
- **wo_id stable across the whole operation.** Reporting, customer-facing references, contractor invoicing all use one identifier.

5.2 Transitions through the phase axis

From phase	To phase	Trigger	Job type after	Status after
NULL	DISCONNECT	wo-shifting-enter-disco nnect worker (right after wo-validate-create)	unchanged (GSD or HSD)	IN_PROGRESS
DISCONNECT	RECONNECT	wo-shifting-enter-reco nnect worker (after disconnect-phase finalize)	GSD → GSR or HSD → HSR	IN_PROGRESS
RECONNECT	NULL	wo-shifting-finalize-r econnect-phase worker (terminal)	unchanged (GSR or HSR)	COMPLETED

Each transition writes a row to `wo_status_history` with the previous/new phase columns populated and emits a `WorkOrderPhaseTransitioned` Kafka event (framework event from DD_WO-01-FRAMEWORK §7).

5.3 The job_type_code change

The job type code on the WO row changes between phases. This is unusual — most WOs have one job type for life. Shifting is the v1.0 exception. The `wo-shifting-enter-reconnect` worker updates `job_type_code` in the same transaction as the phase transition:

- GSD → GSR for GPON shifting.
- HSD → HSR for HFC shifting.

The network type stays the same (GPON or HFC) across the transition; only the disconnect/reconnect suffix changes.

This matters because `wo_finalization_requirements` lookups are keyed on (`operator_code`, `kind`, `job_type_code`). When the disconnect phase finalizes, the framework reads the GSD/HSD row (no notes/attachments required beyond `dispatcher_note` + summary). When the reconnect phase finalizes, the framework reads the GSR/HSR row (full install-like checklist). Same `wo_id`, same row — different requirements at each phase boundary, because the WO has **become** a reconnect WO by then.

Future enhancement: if other flows need similar mid-flight job-type changes, the framework could promote this to a first-class primitive. v1.0 keeps it as a flow-internal detail.

5.4 What if reconnect can't happen?

If the disconnect phase finalizes (account → INA, customer detached from old HomePass) but the reconnect phase can't complete (new address not serviceable, customer disappears, contractor never schedules), the WO sits in `current_phase=RECONNECT`, `status=IN_PROGRESS` indefinitely. This is operationally bad — the customer is paying for service they're not getting.

The flow does not auto-resolve this. Two operational responses are available:

- **Cancel the WO** via POST `/api/work-orders/{woId}/cancel`. The WO transitions to CANCELLED. Operations follow up: either reactivate the customer at the old address (manual intervention through ILM-CFG-01) or terminate the customer (future OSR-DISCONNECT flow).
- **Finalize with NEW_ADDRESS_NOT_SERVICEABLE** as the final reason. Customer's account stays in INA. Retention (via Kafka subscription to `WorkOrderShiftingCompleted` filtering on this final reason) opens a separate conversation with the customer.

Neither response involves un-doing the disconnect-phase finalization. Once an account goes to INA, restoring it to ACT requires a separate, deliberate action — never an implicit "undo" inside the WO module.

6. Account-status side-effects

Two side-effects fire as part of a Shifting WO's lifecycle. Both are emitted as event-payload fields for ILM-CFG-01 (Customer Service module) to consume. The WO module does NOT directly UPDATE the customer-account record — separation of concerns.

6.1 At disconnect-phase finalize

The `wo-shifting-finalize-disconnect`-phase worker emits a `WorkOrderPhaseFinalized` event (flow-specific, §8) with payload field `accountSubStatusChange = {to: "INA", reason: "SHIFTING_DISCONNECT"}`.

ILM-CFG-01 consumes and updates `customer_account.sub_status`.

The worker also emits a `WorkOrderHomePassDetachment` event (flow-specific) with payload `{homepassId: <OLD>, customerId, detachedAt}`. RLM-CFG-01 consumes and clears the old HomePass's `customer_id` field (the HomePass row stays — it's a physical address record — but the customer association is removed). The old HomePass becomes available for a new customer in the future.

6.2 At reconnect-phase finalize

The `wo-shifting-finalize-reconnect-phase` worker emits a `WorkOrderPhaseFinalized` event with payload `accountSubStatusChange = {to: "STF", reason: "SHIFTING_RECONNECT"}`. ILM-CFG-01 consumes and updates the customer account.

Then `wo-emit-homepass-update-at-finalize` emits a `WorkOrderHomePassTopologyResolved` event for the NEW HomePass (carrying captured or confirmed topology fields). RLM-CFG-01 writes the topology to the new HomePass record and attaches the customer to it.

This separation — account-status events go to ILM-CFG-01, HomePass-update events go to RLM-CFG-01 — keeps each downstream module owning its own data without the WO module needing to know any of them directly. The WO module just emits events.

6.3 Failure modes

If ILM-CFG-01 misses the event (downstream outage, DLQ), the customer's account stays in its previous status. Operations recovery: the framework's events are Kafka with at-least-once semantics; ILM-CFG-01's consumer should be idempotent on `(woId, phase)`. For manual recovery, the flow exposes a replay endpoint `POST /api/work-orders/{woId}/phase-events-replay` (§9.2) that re-emits the phase-finalize events for the specified phase.

Same goes for RLM-CFG-01 missing the HomePass detachment / topology-resolution events — re-emit via the replay endpoint.

7. Flow-specific workers

Five workers live in this flow on top of the framework's generic ones and the Installation flow's shared ones (which it reuses):

7.1 wo-shifting-enter-disconnect

External task topic: `wo-shifting-enter-disconnect`.

Runs right after `wo-validate-create`. Action:

1. Set `current_phase = DISCONNECT` on the WO row.
2. Emit `WorkOrderPhaseTransitioned` with `previousPhase=null`, `newPhase=DISCONNECT`.
3. Complete the Camunda task.

Lock duration: 3s.

7.2 wo-shifting-enter-reconnect

External task topic: `wo-shifting-enter-reconnect`.

Runs after the disconnect phase's `wo-shifting-finalize-disconnect-phase` worker completes. Action:

1. Update `job_type_code` on the WO row from `GSD` → `GSR` (or `HSD` → `HSR`) per `network_type`.
2. Set `current_phase = RECONNECT`.
3. Emit `WorkOrderPhaseTransitioned` with `previousPhase=DISCONNECT`, `newPhase=RECONNECT`.
4. Audit-log the `job_type` change.
5. Complete the Camunda task.

Lock duration: 3s.

7.3 wo-shifting-disconnect-prepare

External task topic: wo-shifting-disconnect-prepare.

Runs inside the DISCONNECT subprocess. Action:

1. Validate that the WO has a dispatcher_note capturing the shifting circumstances (customer relocating from X to Y, scheduled date).
2. Validate that the WO has a new_address note with a valid newHomepassId. (This is captured at creation time; if missing, fail with 400 — the agent should have selected the new address before raising the shifting.)
3. Mark the OLD HomePass record as "vacating" via a lightweight event (WorkOrderHomePassVacating, internal to RLM-CFG-01's subscription).
4. Complete the Camunda task.

If validation fails, the worker fails the task with an error code that routes the BPMN to a user task for the dispatcher to correct the inputs before retrying.

Lock duration: 5s.

7.4 wo-shifting-finalize-disconnect-phase

External task topic: wo-shifting-finalize-disconnect-phase.

Runs at the end of the DISCONNECT subprocess. Action:

1. Verify the framework's wo-enforce-finalize-checklist has passed for the GSD/HSD job_type_code (required notes: dispatcher_note + shifting_disconnect_summary + final_reason_set).
2. Emit WorkOrderPhaseFinalized (flow event, §8) with:
 - phase = DISCONNECT
 - accountSubStatusChange = {to: "INA", reason: "SHIFTING_DISCONNECT"}
 - oldHomepassId (read from WO)
3. Emit WorkOrderHomePassDetachment (flow event, §8) with {homepassId: <OLD>, customerId, detachedAt}.
4. Complete the Camunda task.

The worker does NOT transition WO status — status=IN_PROGRESS continues into the RECONNECT phase. Only current_phase changes (via the next worker, wo-shifting-enter-reconnect).

Lock duration: 5s.

7.5 wo-shifting-finalize-reconnect-phase

External task topic: wo-shifting-finalize-reconnect-phase.

Runs at the end of the RECONNECT subprocess, after the framework's checklist has passed. Action:

1. Transition status FINALIZATION_PENDING → COMPLETED (this is the WO's terminal transition).
2. Set completed_at = NOW().
3. Emit WorkOrderPhaseFinalized with:
 - phase = RECONNECT
 - accountSubStatusChange = {to: "STF", reason: "SHIFTING_RECONNECT"}
 - newHomepassId (read from WO)
4. Emit framework's workOrderFinalized (in addition to the phase event — the WO has completed).
5. Complete the Camunda task.

Lock duration: 5s.

7.6 Reused workers from the Installation flow

The RECONNECT phase reuses:

- wo-mark-pattern-b-reroute — same as Installation §7.1. When the Drools client-up-decision returns CLIENT_NOT_UP_REROUTE, the WO is reassigned to maintenance or construction (within the RECONNECT phase,

same wo_id). On return-to-reconnect-contractor, work resumes.

- wo-capture-equipment-bindings-at-finalize — same as Installation §7.2. Validates equipment bindings against per-Job-Type rules. Note: shifting reconnect typically reuses existing customer equipment (the customer brought their modem/ONT to the new address) — the action in wo_equipment_ref_stub rows for these bindings is REUSED rather than INSTALLED. If new equipment is installed (modem damaged in transit, customer upgrading), action is INSTALLED or REPLACED.
- wo-emit-homepass-update-at-finalize — same as Installation §7.4. Reads the topology capture or confirmation note, emits WorkOrderHomePassTopologyResolved for the NEW HomePass. RLM-CFG-01 writes the topology and attaches the customer to the new HomePass.

8. Flow-specific events

This flow extends the framework's event catalog (DD_WO-01-FRAMEWORK §7) with four new events. All on the sophix.work-orders.* topic family. The Installation flow's three events (WorkOrderReroutedPatternB, WorkOrderEquipmentBindingsRecorded, WorkOrderHomePassTopologyResolved) are also fired by this flow when the relevant primitives engage during the RECONNECT phase.

8.1 Event table

Event type	Topic	Emitted when	Key consumers
WorkOrderPhaseTransitioned (framework event, fired by this flow)	sophix.work-orders.phase-transitioned	NULL→DISCONNECT and DISCONNECT→RECONNECT transitions	NOC dashboards, audit, SLA tracker
WorkOrderPhaseFinalized	sophix.work-orders.phase-finalized	DISCONNECT-phase finalize AND RECONNECT-phase finalize	ILM-CFG-01 (account sub_status update), audit
WorkOrderHomePassDetachment	sophix.work-orders.homepass-detachment	DISCONNECT-phase finalize	RLM-CFG-01 (clears customer_id from old HomePass), audit
WorkOrderHomePassTopologyResolved (Installation event, fired by this flow)	sophix.work-orders.homepass-topology-resolved	RECONNECT-phase finalize — topology captured or confirmed for the NEW HomePass	RLM-CFG-01 (updates new HomePass record), audit
WorkOrderShiftingCompleted	sophix.work-orders.shifting-completed	After WorkOrderFinalized for a SHIFTING-kind WO, with both phase finalizations and topology resolution complete	Finance (contractor invoicing per shifting visit), retention (subscribes for NEW_ADDRESS_NOT_SERVICEABLE final reason), audit

8.2 Sample event — WorkOrderPhaseFinalized (DISCONNECT)

```
{
  "eventType": "WorkOrderPhaseFinalized",
  "aggregateId": "wo_01HZX9K8M7Q2N4P6R3T5W8Y0Z2",
  "timestamp": "2026-05-18T10:30:00.512Z",
  "payload": {
    "woId": "wo_01HZX9K8M7Q2N4P6R3T5W8Y0Z2",
    "operatorCode": "WIK",
    "kind": "SHIFTING",
    "jobTypeCode": "GSD",
    "networkType": "GPON",
    "phase": "DISCONNECT",
    "customerId": "cust_01HX...",
    "oldHomepassId": "hp_01HX_OLD...",
    "newHomepassId": "hp_01HX_NEW...",
    "accountSubStatusChange": {
      "to": "INA",
      "reason": "SHIFTING_DISCONNECT"
    },
    "finalReasonCode": "DISCONNECTED_REMOTELY",
    "finalizedBy": "csr_user_42",
    "finalizedByRole": "CSR",
    "finalizedAt": "2026-05-18T10:30:00.123Z"
  }
}
```

8.3 Sample event — WorkOrderPhaseFinalized (RECONNECT)

```
{
  "eventType": "WorkOrderPhaseFinalized",
  "aggregateId": "wo_01HZX9K8M7Q2N4P6R3T5W8Y0Z2",
  "timestamp": "2026-05-19T15:45:00.123Z",

```

```

"payload": {
  "woId": "wo_01HZX9K8M7Q2N4P6R3T5W8Y0Z2",
  "operatorCode": "WIK",
  "kind": "SHIFTING",
  "jobTypeCode": "GSR",
  "networkType": "GPON",
  "phase": "RECONNECT",
  "customerId": "cust_01HX...",
  "oldHomepassId": "hp_01HX_OLD...",
  "newHomepassId": "hp_01HX_NEW...",
  "accountSubStatusChange": {
    "to": "STF",
    "reason": "SHIFTING_RECONNECT"
  },
  "finalReasonCode": "RECONNECTED_AT_NEW_ADDRESS",
  "finalizedBy": "ctr_Z071H_dispatcher_01",
  "finalizedByRole": "CONTRACTOR",
  "finalizedAt": "2026-05-19T15:45:00.000Z"
}
}

```

8.4 Sample event — WorkOrderHomePassDetachment

```

{
  "eventType": "WorkOrderHomePassDetachment",
  "aggregateId": "wo_01HZX9K8M7Q2N4P6R3T5W8Y0Z2",
  "timestamp": "2026-05-18T10:30:01.000Z",
  "payload": {
    "woId": "wo_01HZX9K8M7Q2N4P6R3T5W8Y0Z2",
    "operatorCode": "WIK",
    "homepassId": "hp_01HX_OLD...",
    "customerId": "cust_01HX...",
    "detachedAt": "2026-05-18T10:30:00.123Z",
    "reason": "SHIFTING_DISCONNECT_FINALIZED"
  }
}

```

8.5 Sample event — WorkOrderShiftingCompleted

```

{
  "eventType": "WorkOrderShiftingCompleted",
  "aggregateId": "wo_01HZX9K8M7Q2N4P6R3T5W8Y0Z2",
  "timestamp": "2026-05-19T15:45:30.000Z",
  "payload": {
    "woId": "wo_01HZX9K8M7Q2N4P6R3T5W8Y0Z2",
    "operatorCode": "WIK",
    "kind": "SHIFTING",
    "networkType": "GPON",
    "customerId": "cust_01HX...",
    "oldHomepassId": "hp_01HX_OLD...",
    "newHomepassId": "hp_01HX_NEW...",
    "disconnectFinalizedAt": "2026-05-18T10:30:00.123Z",
    "reconnectFinalizedAt": "2026-05-19T15:45:00.000Z",
    "totalDurationSeconds": 105299,
    "finalReasonCode": "RECONNECTED_AT_NEW_ADDRESS",
    "reconnectContractorId": "ctr_Z071H",
    "reconnectTeamId": "team_rapid_response",
    "reconnectTechnicianId": "tech_01HZ_LANGATA_04",
    "patternBLoopCount": 0,
    "equipmentBindings": [
      {
        "equipmentRefId": "eqref_01HZ_R1",
        "equipmentKind": "GPON_ONT",
        "serialNumber": "GPON-ONT-ABC789",
        "action": "REUSED"
      }
    ],
    "topologyResolution": {
      "mode": "CONFIRMED",
      "confirmedAsPlanned": true,
      "corrections": null
    }
  }
}

```

Finance subscribes to this event for contractor invoicing. Retention subscribes (with filter `finalReasonCode == "NEW_ADDRESS_NOT_SERVICEABLE"`) to follow up with customers whose shifting failed at the new address.

9. REST API additions

This flow uses the framework's REST surface (DD_WO-01-FRAMEWORK §8) plus the Installation flow's /equipment-bindings and /topology-replay endpoints. One additional endpoint:

9.1 POST /api/work-orders/{woId}/phase-events-replay — Operations recovery

Operations recovery endpoint. If ILM-CFG-01 or RLM-CFG-01 missed the phase-finalize events (downstream outage), an operations engineer can replay them:

```
POST /api/work-orders/{woId}/phase-events-replay HTTP/1.1
{
  "phase": "DISCONNECT",
  "replayReason": "ILM-CFG-01 consumer lag - recovering from DLQ",
  "replayedBy": "ops_engineer_03"
}
```

Framework verifies the phase was actually finalized on this WO; re-emits `WorkOrderPhaseFinalized` for the specified phase (and `WorkOrderHomePassDetachment` if phase=DISCONNECT). Consumer-side idempotency on (woId, phase) is assumed. Audit-logged.

Errors:

Status	When
400 PHASE_NOT_FINALIZED	The specified phase has not yet finalized on this WO (e.g., asking to replay RECONNECT on a WO still in DISCONNECT phase)
400 INVALID_PHASE	Phase value not in {DISCONNECT, RECONNECT}
404 WO_NOT_FOUND	wo_id doesn't exist or is not a SHIFTING kind

10. Workshop sources

The Wananchi KE Shifting workflow (workshop 2026-04-29 + Bildad Gitonga transcript 2026-04-30) requires:

- A two-leg operation per shifting: Disconnect (GSD for GPON, HSD for HFC) at the old address, then Reconnect (GSR/HSR) at the new address.
- The Disconnect leg is system-driven, no site visit, work units = 0. Triggers `sub_status` → INA on the customer account and removes the customer from the old HomePass.
- The Reconnect leg requires a full site visit (work units = 1), schedules a team, captures the new address in Notes, requires optical/signal check before finalization, triggers `sub_status` → STF, and updates the HomePass to the new location.
- Finalization order is strict: Disconnect must finalize before Reconnect can begin.
- Shifting is free of charge for the customer (no billing coupling).
- Shifting can be raised by CSR or by contractor teams.
- Standard Wananchi finalization discipline applies to the Reconnect leg: required-before-finalize checklist (findings, solution, optical/signal, final reason, photos), 2-step finalize, Pattern B reroute available if the new address can't be brought up due to signal/infrastructure issues.

Cross-country variation. In Senegal, the Reconnect leg may involve last-mile deployment (running fiber from a PBO to the new premise). The tech captures topology via "app terrain" — same REST API as other clients. The variation is supported by per-operator note-kind registration + per-operator finalization requirements (§3.6); no code change between KE and SN models.

11. Cross-DD coordination

11.1 Module owner

This satellite is owned by the Fulfillment & Operations team alongside its parent DD (WO module). It depends on DD_WO-01-FRAMEWORK-v1.0 for all primitives and reuses workers + endpoints from DD_WO-01-FLOW-INSTALLATION-v1.0 (Pattern B reroute, equipment binding capture, HomePass topology resolution).

11.2 Impacted modules

Module	Direction	Contract
WO-01-FRAMEWORK	this flow ← framework	Consumes all framework primitives: state machine, REST API, multi-phase support, Camunda integration, schemas, generic workers, Kafka events.
WO-01-FLOW-INSTALLATION	this flow ← Installation flow	Reuses the Pattern B reroute worker, equipment binding capture, and HomePass topology resolution worker. The RECONNECT phase's middle section is intentionally identical to the Installation flow's on-site work loop.
ILM-CFG-01 (Customer Service — future)	this flow → ILM-CFG-01	Consumes <code>WorkOrderPhaseFinalized</code> to update <code>customer_account.sub_status</code> (INA at disconnect, STF at reconnect). v1.0 uses <code>customer_stub</code> .
RLM-CFG-01 (HomePass)	this flow ↔ RLM	Reads <code>new_homepass.tech_region_code</code> for reconnect-phase assignment. At disconnect-finalize, consumes <code>WorkOrderHomePassDetachment</code> to clear <code>customer_id</code> from the OLD HomePass. At reconnect-finalize, consumes <code>WorkOrderHomePassTopologyResolved</code> to write the captured/confirmed topology onto the NEW HomePass and attach the customer.
BO UI (CSR Console)	BO UI → this flow	CSR creates shifting WOs (selects new address, fills in dispatcher note, schedules reconnect). Confirms disconnect phase.
Contractor BO UI	BO UI → this flow	Contractor dispatcher claims reconnect-phase WOs, schedules tech, finalizes after on-site work. Same UI as installation flow.
Future Inventory / Equipment Catalog	this flow → Inventory	<code>WorkOrderEquipmentBindingsRecorded</code> events (reused from Installation flow's primitive) consumed by Inventory once it ships — for shifting these often have <code>action=REUSED</code> (customer brought equipment to new address).
Finance	this flow → Finance	<code>WorkOrderShiftingCompleted</code> consumed for contractor invoicing per shifting visit (separate contractor fee schedule from installs).
Retention	this flow → Retention	Subscribes to <code>WorkOrderShiftingCompleted</code> filtering on <code>finalReasonCode == "NEW_ADDRESS_NOT_SERVICEABLE"</code> to spawn retention conversations.

11.3 Boundary clarifications

What's in scope for this satellite vs what's not:

- ■ Shifting BPMN (single-WO two-phase), Drools KJAR for reconnect-phase routing + client-up gate, operator-config seeds, phase transition mechanics, account-status side-effects, new-address capture, HomePass detachment + attachment events.
- ■ The WO module's state machine, REST API surface, schemas, Camunda primitives — owned by DD_WO-01-FRAMEWORK.
- ■ Installation flow — owned by DD_WO-01-FLOW-INSTALLATION.
- ■ Support flow — owned by future DD_WO-01-FLOW-SUPPORT.
- ■ Customer termination / suspension / churn — future OSR-DISCONNECT flow.
- ■ Network migration (HFC ↔ GPON in place) — future flow.
- ■ Customer billing for shifting — shifting is free; no BIL-* coupling.
- ■ Account record updates — emitted via events, owned by ILM-CFG-01.

11.4 Dependencies (build order)

1. **DD_WO-01-FRAMEWORK-v1.0** must be shipped before this satellite.
2. **DD_WO-01-FLOW-INSTALLATION-v1.0** should be shipped before or alongside — this flow reuses several of its workers and endpoints.

3. **RLM-CFG-01 (HomePass)** must exist for tech-region lookup, HomePass detachment + topology update consumers.
4. **Drools engine + KJAR deployment** infrastructure (shared with Installation flow).
5. **BO UI CSR Console** for creating shifting WOs.

11.5 Migration approach

When this flow ships:

1. **DDL** — the framework's tables are the only schema; this flow adds no new tables. Seed `wo_flow_config`, `wo_job_type_catalog` (4 new rows: GSD/GSR/HSD/HSR), `wo_note_kind_registry` (2 new rows), `wo_finalization_requirements` (4 rows), `wo_final_reason_catalog` rows per §3.
2. **BPMN deployment** — deploy `WorkOrder_Shifting_WIK.bpmn` to Camunda.
3. **Drools KJAR deployment** — deploy `wo-shifting-wik-v1.0.kjar`.
4. **Cutover** — CSR Console exposes "Raise Shifting WO" workflow; contractors' BO UIs surface shifting WOs in their queues alongside installs and support tickets.
5. **Future operator onboarding (WUG/WTZ/YASSN)** — add operator-suffixed BPMN + KJAR; add per-operator config rows; no code change.

12. Runtime walkthrough — narrative

One scenario: a happy-path GPON shifting from creation to completion, showing both phases and all side-effects.

Scenario. Customer John Otieno is moving from Lavington (NRB-LAVINGTON) to Karen (NRB-KAREN), staying with Wananchi on the same GPON internet package. He calls the CSR on 2026-05-18 morning.

12.1 Creation

Step 1 — CSR creates the WO via BO UI. UI calls:

```
POST /api/work-orders HTTP/1.1
{
  "operatorCode":      "WIK",
  "kind":              "SHIFTING",
  "jobTypeCode":       "GSD",
  "customerId":        "cust_01HY...",
  "homepassId":        "hp_01HX_OLD_LAV",
  "originatingContextType": "BACKOFFICE",
  "originatingContextRef": "csr_user_42",
  "idempotencyKey":     "shift-csr_42-cust_01HY-2026-05-18-09-15",
  "metadata":          {
    "newHomepassIdAtCreation": "hp_01HX_NEW_KAR",
    "customerName":           "John Otieno",
    "contactNumber":          "+254712345678"
  }
}
```

Framework: writes `work_order` row with `status=PENDING`, `kind=SHIFTING`, `jobTypeCode=GSD`, `homepass_id=hp_01HX_OLD_LAV`, `metadata.newHomepassIdAtCreation=hp_01HX_NEW_KAR`. Starts `WorkOrder_Shifting_WIK` Camunda process instance. Returns 201 with `woId=wo_01HZ...SH7`. Emits `WorkOrderCreated`.

Step 2 — CSR adds the new_address note. UI calls:

```
POST /api/work-orders/wo_01HZ...SH7/notes
{
  "noteKind":      "new_address",
  "contentJsonb":  {
    "newHomepassId": "hp_01HX_NEW_KAR",
    "addressFreeText": "Karen, Bogani Road 12, House 7",
    "agentNotes":     "Customer relocating for work. Schedule reconnect for 2026-05-19 afternoon."
  },
  "appendedBy":    "csr_user_42",
  "appendedByRole": "CSR"
}
```

Framework writes the note, emits `WorkOrderNoteAppended`.

Step 3 — CSR adds the dispatcher note for context. UI calls:

```
POST /api/work-orders/wo_01HZ...SH7/notes
```

```
{ "noteKind": "dispatcher_note", "contentText": "GPON shifting. Customer relocating from Lavington to Karen. Internet package
```

12.2 DISCONNECT phase

Step 4 — BPMN runs wo-shifting-enter-disconnect. Framework worker sets `current_phase=DISCONNECT`. Status remains `PENDING` (transitions to `IN_PROGRESS` when the actual disconnect work begins, after the prepare step). Emits `WorkOrderPhaseTransitioned` `{previousPhase: null, newPhase: DISCONNECT}`.

Step 5 — BPMN runs wo-shifting-disconnect-prepare. Validates `dispatcher_note` ✓, `new_address` note with valid `newHomepassId` ✓. Emits internal `WorkOrderHomePassVacating` event (RLM-CFG-01 marks the OLD HomePass as having a vacating customer — used for capacity planning).

Step 6 — BPMN parks at user task await-csr-confirm-disconnect. CSR confirms via:

```
POST /api/work-orders/wo_01HZ...SH7/finalize-first-confirm
{
  "finalReasonCode": "DISCONNECTED_REMOTELY",
  "confirmedBy":    "csr_user_42"
}
```

Status transitions `PENDING` → `FINALIZATION_PENDING` via framework's first-confirm semantics. Wait — there's a wrinkle: the WO went from `PENDING` to `FINALIZATION_PENDING` directly, skipping `IN_PROGRESS`, because GSD has `requires_site_visit = false`. The flow's BPMN handles this by having the user task accept first-confirm directly from `PENDING` (the framework's status state machine permits this for no-site-visit flows). The `wo-shifting-disconnect-prepare` worker can also push the status to `IN_PROGRESS` briefly to make the transition explicit; v1.0 chooses the brief-`IN_PROGRESS` path for audit clarity.

Re-stating: after `wo-shifting-disconnect-prepare`, status is briefly set to `IN_PROGRESS` (via internal call), then CSR's first-confirm moves it to `FINALIZATION_PENDING`. `wo_status_history` shows `PENDING` → `IN_PROGRESS` → `FINALIZATION_PENDING` within seconds. CSR adds a `shifting_disconnect_summary` note before first-confirm:

```
POST /api/work-orders/wo_01HZ...SH7/notes
{ "noteKind": "shifting_disconnect_summary", "contentJsonb": { "actionTaken": "PROVISIONING_DISABLED_REMOTELY" } }
```

Step 7 — CSR second-confirms the disconnect. UI calls:

```
POST /api/work-orders/wo_01HZ...SH7/finalize-second-confirm
```

Framework's `wo-enforce-finalize-checklist` reads `wo_finalization_requirements` for (WIK, SHIFTING, GSD): required notes `[dispatcher_note, shifting_disconnect_summary, final_reason_set]` — all present. Pass.

But: instead of status → `COMPLETED` (the framework's default second-confirm behavior), the BPMN routes the second-confirm into the flow's `wo-shifting-finalize-disconnect-phase` worker. This worker DOESN'T transition status to `COMPLETED`; it only finalizes the phase. Status returns to `IN_PROGRESS` for the next phase. The framework primitive supports this — the second-confirm is a Camunda message that the flow's BPMN intercepts before the default status transition fires.

Step 8 — wo-shifting-finalize-disconnect-phase runs. Emits:

- `WorkOrderPhaseFinalized` with `phase=DISCONNECT`, `accountSubStatusChange={to:"INA", reason:"SHIFTING_DISCONNECT"}`, `oldHomepassId=hp_01HX_OLD_LAV`, `newHomepassId=hp_01HX_NEW_KAR` (sample 8.2)
- `WorkOrderHomePassDetachment` with `{homepassId: hp_01HX_OLD_LAV, customerId: cust_01HY..., detachedAt: 2026-05-18T10:30:00Z}` (sample 8.4)

ILM-CFG-01 consumes the phase-finalized event, updates `customer_account.sub_status = INA`. RLM-CFG-01 consumes the detachment event, clears `customer_id` from `hp_01HX_OLD_LAV`.

12.3 RECONNECT phase

Step 9 — BPMN runs wo-shifting-enter-reconnect. Updates `job_type_code` from `GSD` → `GSR`. Sets `current_phase=RECONNECT`. Status stays `IN_PROGRESS`. Emits `WorkOrderPhaseTransitioned` `{previousPhase: DISCONNECT, newPhase: RECONNECT}`. The WO is now effectively a "GSR reconnect at new address" WO from this point forward.

Step 10 — BPMN runs wo-auto-assign-contractor. Drools KJAR's `contractor-assignment-decision` runs with `{jobTypeCode=GSR, networkType=GPON, techRegionCode=NRB-KAREN (from NEW HomePass)}`. Rule `r3-gpon-by-region` fires: `ctr_Z071H + team_rapid_response`, `TEAM` mode. Framework: row updates, emits `WorkOrderAssigned`.

Step 11 — Notification to team. tech_01HZ_KAREN_07 **claims via** POST /claim. Framework validates: tech belongs to team ✓, schedule covers Tue afternoon ✓, no other pending WO ✓. Atomic update wins. Emits WorkOrderClaimedByTechnician.

Step 12 — Tech arrives at new address on 2026-05-19. Contractor dispatcher calls /start. Status was IN_PROGRESS already (didn't go back to ASSIGNED — the assignment within RECONNECT just set the contractor/team/tech fields; the framework's R-WO-FW-19 says *_at columns set exactly once, so in_progress_at is unchanged from when it was first set during the disconnect phase). The contractor dispatcher's call to /start is now effectively a no-op on status (it's already IN_PROGRESS) but it does emit WorkOrderInProgress again for audit clarity, with metadata.phase=RECONNECT.

Wait — WorkOrderInProgress is documented as a one-time event in the framework. For phase 2 entry, the more appropriate signal is the WorkOrderPhaseTransitioned event already emitted in step 9. The framework's /start API on a WO that's already IN_PROGRESS returns 200 idempotently without re-emitting. v1.0 codifies this: tech-arrival in a multi-phase WO's later phase is signaled by wo_audit_log.action = 'phase_started' with phase=RECONNECT, not by a new state event.

Step 13 — Tech captures notes + attachments + equipment binding. Same calls as Installation §12.1 step 5, but with shifting-specific values:

```
POST .../notes noteKind=findings, contentText="ONT moved from old premises and reinstalled at new address. Splice on exis
POST .../notes noteKind=solution, contentText="Standard reconnect at new HomePass. Customer up."
POST .../notes noteKind=optical_readings, contentJsonb={"oltPort":"0/3/2/4", "onuId":12, "ontRxDbm":-22.5, "ontTxDbm":2.1, "o
POST .../notes noteKind=client_up_evidence, contentJsonb={"evidenceType":"OPTICAL_READING_PHOTO"}
POST .../notes noteKind=network_topology_confirmation, contentJsonb={"confirmedAsPlanned":true}
POST .../attachments category=setup_photo, file=ont_mounted_karen.jpg
POST .../attachments category=rf_optical_reading, file=easydocsis_karen.png
POST .../equipment-bindings
  bindings:[{equipmentKind:"GPON_ONT", brand:"Huawei", model:"HG8245H", serialNumber:"GPON-ONT-ABC789", action:"REUSED"}]
```

Note action=REUSED — the customer brought their existing ONT from the old address. Framework records the binding in wo_equipment_ref_stub and emits WorkOrderEquipmentBindingsRecorded.

Step 14 — Contractor dispatcher first-confirms.

```
POST /api/work-orders/wo_01HZ...SH7/finalize-first-confirm
{"finalReasonCode":"RECONNECTED_AT_NEW_ADDRESS", "confirmedBy":"ctr_Z071H_dispatcher_01"}
```

Status FINALIZATION_PENDING. BPMN advances to client-up Drools gateway.

Step 15 — client-up-decision runs. Optical reading -22.5 dBm ∈ [-28, -8] ✓. Returns CLIENT_UP. Equipment-bindings validation passes (REUSED is a valid action; GPON_ONT present).

Step 16 — Second-confirm.

```
POST /api/work-orders/wo_01HZ...SH7/finalize-second-confirm
```

Framework's checklist reads wo_finalization_requirements for (WIK, SHIFTING, GSR): required notes [findings, solution, optical_readings, client_up_evidence, new_address, network_topology_confirmation, final_reason_set] ✓; attachments [setup_photo, rf_optical_reading] ✓. Pass.

BPMN advances to wo-shifting-finalize-reconnect-phase worker.

Step 17 — wo-shifting-finalize-reconnect-phase runs. Status FINALIZATION_PENDING → COMPLETED. completed_at set. Emits:

- Framework WorkOrderFinalized
- Flow WorkOrderPhaseFinalized with phase=RECONNECT, accountSubStatusChange={to:"STF", reason:"SHIFTING_RECONNECT"}, newHomepassId=hp_01HX_NEW_KAR (sample 8.3)

ILM-CFG-01 consumes the phase-finalized event, updates customer_account.sub_status = STF (back to active).

Step 18 — wo-emit-homepass-update-at-finalize runs. Reads network_topology_confirmation note. confirmedAsPlanned=true, no corrections. Emits WorkOrderHomePassTopologyResolved with topologyMode=CONFIRMED_AS_PLANNED, homepassId=hp_01HX_NEW_KAR. RLM-CFG-01 consumes: no topology change (planning matches reality), but attaches the customer to hp_01HX_NEW_KAR — homepass.customer_id = cust_01HY....

Step 19 — wo-emit-shifting-completed runs. Emits flow's WorkOrderShiftingCompleted rich event (sample 8.5). Finance picks this up for contractor invoicing per shifting visit. Retention sees finalReasonCode=RECONNECTED_AT_NEW_ADDRESS and ignores it (would only act on NEW_ADDRESS_NOT_SERVICEABLE).

Step 20 — BPMN end. Process instance closes.

Total elapsed: ~30 hours (CSR creates 2026-05-18 09:15, disconnect finalizes 2026-05-18 10:30 same day, reconnect finalizes 2026-05-19 15:45 the following afternoon). Customer was in INA status for ~29 hours during the move — a target the future SLA engine (Phase 2) will measure as "shifting INA window".

wo_status_history shows: PENDING → IN_PROGRESS → FINALIZATION_PENDING → IN_PROGRESS → FINALIZATION_PENDING → COMPLETED. Five transitions across two phases. wo_assignment_history shows one row (the reconnect-phase auto-assignment to ctr_Z071H + tech claim).

End of DD_WO-01-FLOW-SHIFTING-v1.0.